

EcoTroc

Projet Numérique Durable TI616

Équipe de 5 personnes

2026

Table des matières

1	Présentation du projet	3
2	Architecture et conception	3
2.1	Architecture générale	3
2.2	Diagramme de cas d'utilisation	4
2.3	Diagramme de classes	5
2.4	Diagramme de séquence	6
2.5	Base de données	7
2.6	Optimisation BDD	7
3	Choix technologiques	7
4	Implémentation	7
4.1	Implémentation des opérations CRUD	7
4.1.1	CRUD Utilisateurs	7
4.1.2	CRUD Annonces	8
4.1.3	Optimisations des requêtes	9
4.1.4	Sécurité des opérations	9
4.1.5	Performances	9
4.2	Fonctionnalités	9
4.3	Accès base de données	9
4.4	Gestion des données	10
5	Analyse de l'empreinte carbone	10
5.1	Outils utilisés	10
5.2	Résultats	10
6	Tests et validation	10
6.1	Tests fonctionnels	10
6.2	Tests de performance	10
6.3	Sécurité	11
7	Organisation de l'équipe	11
8	Conclusion	11

9	Outils de test	12
9.1	EcoIndex	12
9.2	Lighthouse	13
9.3	Page Speed Insights	14
9.4	CatchPoint	15

1 Présentation du projet

EcoTroc est une plateforme de troc écoresponsable permettant aux utilisateurs d'échanger des objets ou des points.

Objectif : Réduire la consommation en favorisant la réutilisation via une plateforme sobre.

Utilisateurs cibles :

- Étudiants
- Particuliers

Contraintes Green IT :

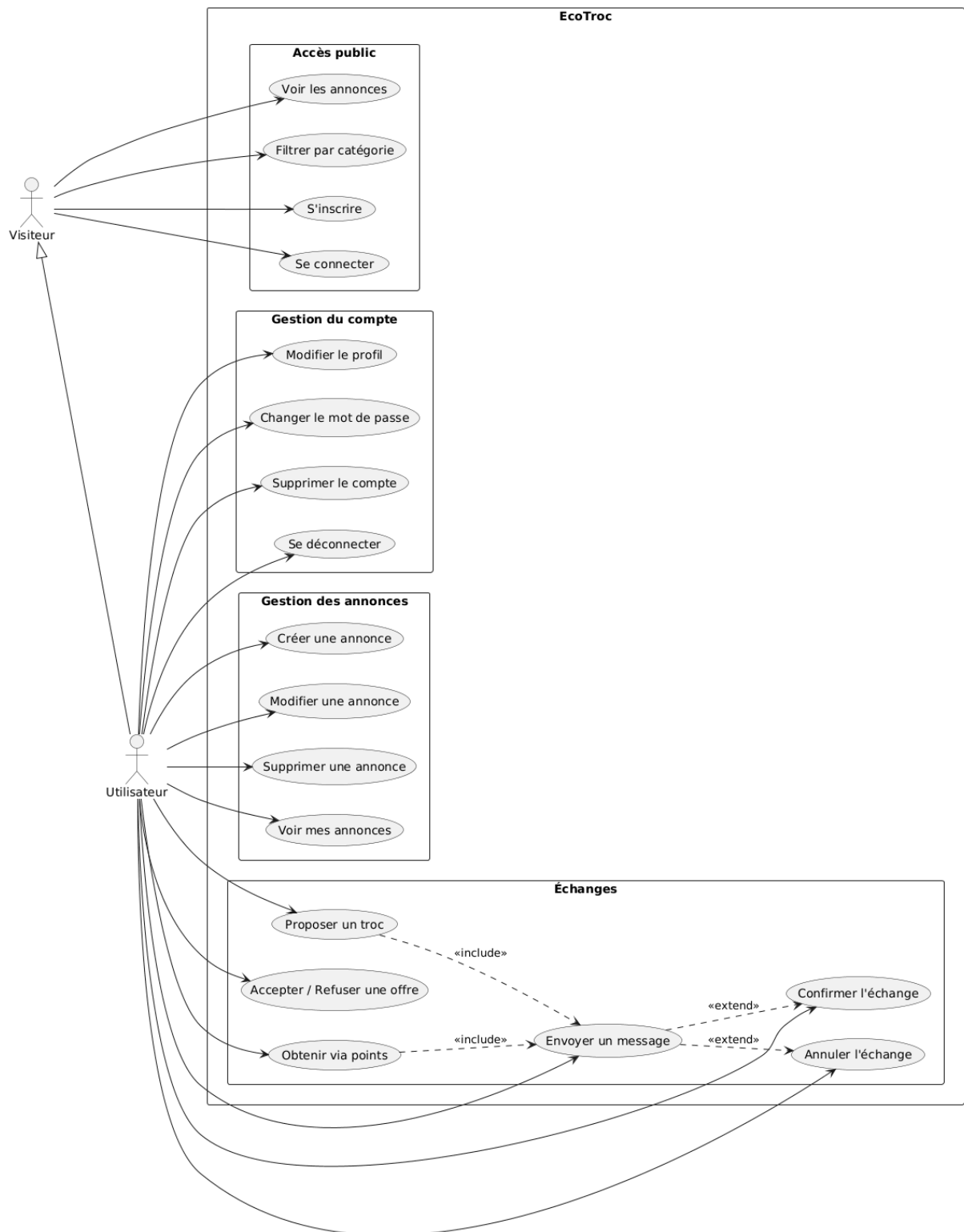
- Limiter le poids des pages
- Minimiser les requêtes HTTP
- Réduire les dépendances

2 Architecture et conception

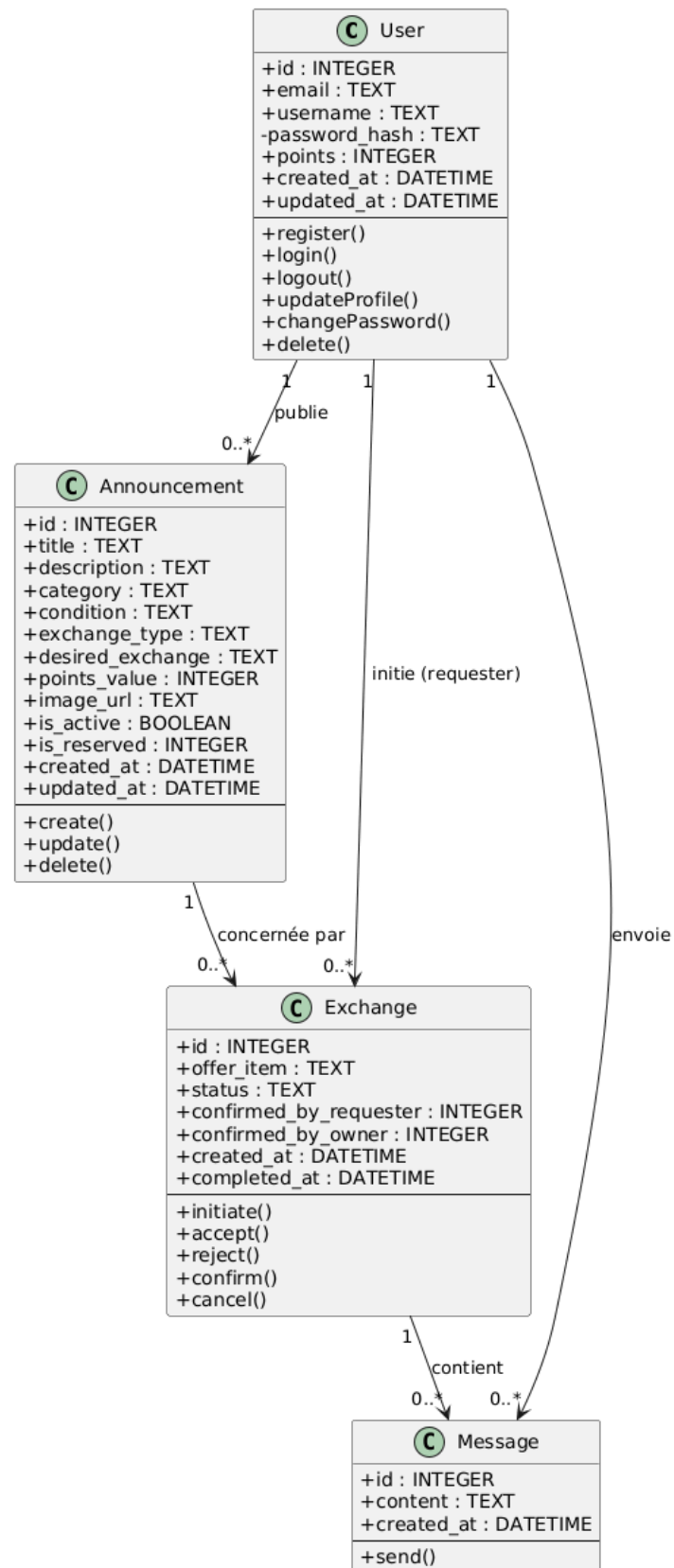
2.1 Architecture générale

- Frontend : HTML / CSS / JS natif
- Backend : Node.js + Express
- Base de données : Turso (LibSQL)
- Stockage images : Vercel Blob

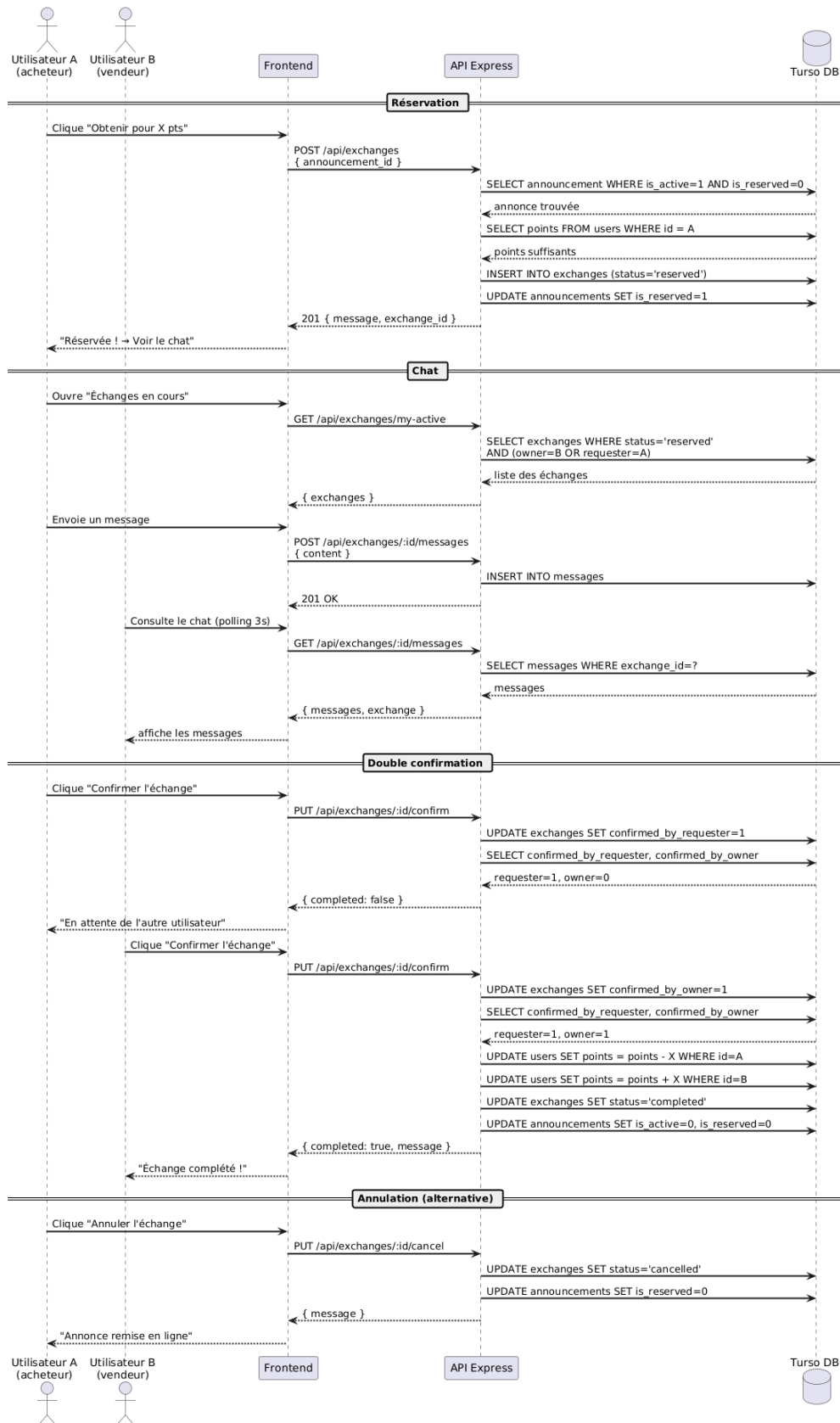
2.2 Diagramme de cas d'utilisation



2.3 Diagramme de classes



2.4 Diagramme de séquence



2.5 Base de données

La base utilise Turso (version modifiée de LibSQL) avec une compatibilité SQLite en local.

Le système implémente :

- Connexion à Turso en production
- Fallback SQLite en local
- Initialisation automatique du schéma
- Gestion de migrations simples

Extrait du fonctionnement :

```
if (TURSO_URL && TURSO_AUTH_TOKEN) {  
  client = createClient({...});  
} else {  
  client = new SQLiteClient(local.db);  
}
```

2.6 Optimisation BDD

- Requêtes paramétrées
- Pagination (LIMIT / OFFSET)
- Normalisation des résultats
- Sélection explicite des colonnes (pas de SELECT *)

3 Choix technologiques

Composant	Technologie	Justification
Frontend	HTML / CSS / JS natif	Léger, sans dépendance
Backend	Node.js + Express	Minimal et performant
Base de données	Turso (LibSQL)	Serverless, compatible SQLite
Images	Vercel Blob	CDN permanent, compression WebP
Auth	bcrypt + sessions	Sécurité standard

4 Implémentation

4.1 Implémentation des opérations CRUD

Le projet EcoTroc implémente des opérations CRUD (Create, Read, Update, Delete) complètes pour deux entités principales : les utilisateurs et les annonces.

Cela permet :

- Une structure simple du code backend
- Une meilleure lisibilité
- Une optimisation des requêtes

De plus, le système supporte deux modes :

- Turso (production)
- SQLite (développement local)

4.1.1 CRUD Utilisateurs

Create — Création d'un utilisateur Lors de l'inscription, les données sont validées côté serveur puis le mot de passe est hashé avant insertion :

```
INSERT INTO users (email, username, password_hash, points)  
VALUES (?, ?, ?, 10)
```

Cela garantit :

- Sécurité (aucun mot de passe en clair)
- Protection contre les injections SQL

Read — Lecture des utilisateurs Deux types de lecture sont implémentés :

- Profil utilisateur :

```
SELECT id, email, username, points
FROM users WHERE id = ?
```
- Liste paginée :

```
SELECT id, username, points
FROM users LIMIT 20 OFFSET ?
```

La pagination permet de limiter la charge serveur et le nombre de données transférées.

Update — Modification La mise à jour permet de modifier le profil ou le mot de passe utilisateur :

```
UPDATE users
SET username = ?, email = ?
WHERE id = ?
```

Une vérification des droits est effectuée avant modification.

Delete — Suppression La suppression d'un utilisateur nécessite une double confirmation côté frontend :

```
DELETE FROM users WHERE id = ?
```

Cette opération est protégée pour éviter toute suppression accidentelle.

4.1.2 CRUD Annonces

Create — Création d'une annonce

```
INSERT INTO announcements
(title, description, category, exchange_type,
 points_value, image_url, user_id)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

Chaque annonce est associée à un utilisateur via une clé étrangère.

Read — Consultation

- Liste paginée (4 par page) :

```
SELECT id, title, category, image_url
FROM announcements
WHERE is_active = 1 AND is_reserved = 0
LIMIT 4 OFFSET ?
```
- Détail :

```
SELECT a.*, u.username
FROM announcements a
JOIN users u ON a.user_id = u.id
WHERE a.id = ?
```

Un filtrage par catégorie est également implémenté.

Update — Modification

```
UPDATE announcements
SET title = ?, description = ?, category = ?
WHERE id = ?
```

Seul le propriétaire de l'annonce peut la modifier.

Delete — Suppression

```
DELETE FROM announcements WHERE id = ?
```

Une vérification d'autorisation est effectuée avant suppression.

4.1.3 Optimisations des requêtes

Plusieurs optimisations ont été mises en place :

- Utilisation de requêtes paramétrées (sécurité)
- Limitation des colonnes sélectionnées (pas de SELECT *)
- Pagination systématique (LIMIT / OFFSET)
- Réduction du nombre de requêtes par page

4.1.4 Sécurité des opérations

Les opérations CRUD respectent les bonnes pratiques :

- Hash des mots de passe avec bcrypt
- Protection contre les injections SQL
- Vérification des droits utilisateur
- Sessions sécurisées

4.1.5 Performances

Les performances sont optimisées grâce à :

- Temps de réponse API inférieur à 200 ms
- Moins de 5 requêtes base de données par page
- Utilisation d'une base serverless (Turso)

4.2 Fonctionnalités

- CRUD utilisateurs (profil, mot de passe, suppression de compte)
- CRUD annonces avec upload d'image
- Compression automatique des images en WebP (~20 Ko via Canvas API)
- Authentification sécurisée (sessions + bcrypt)
- Système d'échange : troc et points
- Réservation : l'annonce disparaît de l'accueil pendant la négociation
- Chat en temps réel entre les deux utilisateurs (polling toutes les 3 s)
- Double confirmation requise pour finaliser un échange
- Annulation possible — l'annonce redevient disponible

4.3 Accès base de données

Fonctions principales :

```
async function run(sql, params)
async function get(sql, params)
async function all(sql, params)
```

Ces fonctions permettent :

- Exécution optimisée
- Centralisation des accès
- Réduction des requêtes

4.4 Gestion des données

- Conversion automatique des types ($\text{BigInt} \rightarrow \text{Number}$)
- Normalisation des résultats SQL

5 Analyse de l’empreinte carbone

5.1 Outils utilisés

- EcoIndex
- Google Lighthouse
- CatchPoint
- Page Speed Insights

5.2 Résultats

Indicateur	Score
Poids	>100 Ko
Requêtes	7
Lighthouse	100

Comme nous avons obtenu des scores presque parfaits dès les premiers tests, nous n’avons pas trouvé pertinent d’optimiser davantage le code.

6 Tests et validation

6.1 Tests fonctionnels

Test	Résultat
Créer utilisateur	OK
Connexion	OK
Modification profil	OK
Changement mot de passe	OK
Suppression compte	OK
CRUD annonces	OK
Upload et compression image	OK
Échange par points	OK
Échange par troc	OK
Chat entre utilisateurs	OK
Confirmation / Annulation	OK
Accès sécurisé	OK

6.2 Tests de performance

- Lighthouse = 100
- FCP = 0.2 s
- LCP = 0.2 s

6.3 Sécurité

- Aucun mot de passe en clair, stockage du hash avec bcrypt
- Protection contre les injections SQL
- Sessions sécurisées avec Express
- Échappement HTML côté client (protection XSS)

7 Organisation de l'équipe

Répartition équilibrée :

- **Julien Casamian – Lead Dev**
 - Architecture backend et frontend
 - Intégration Vercel Blob (images)
 - Système d'échange et chat
- **Eham Bill Abouzi – Frontend**
 - UI HTML / CSS
 - Optimisation Green IT (poids, images)
- **Imrân Benessam – Backend**
 - Routes Express
 - CRUD utilisateurs et annonces
 - Intégration Turso
- **Dorian Anguille – DevOps**
 - Déploiement Vercel
 - Tests Lighthouse / EcoIndex
 - Sécurité
- **Pascal Chen – UX / Design**
 - Maquettes et expérience utilisateur
 - Cohérence visuelle de l'interface

8 Conclusion

Le projet EcoTroc démontre qu'il est possible de créer une application fonctionnelle et complète tout en respectant les principes du Green IT.

Points forts :

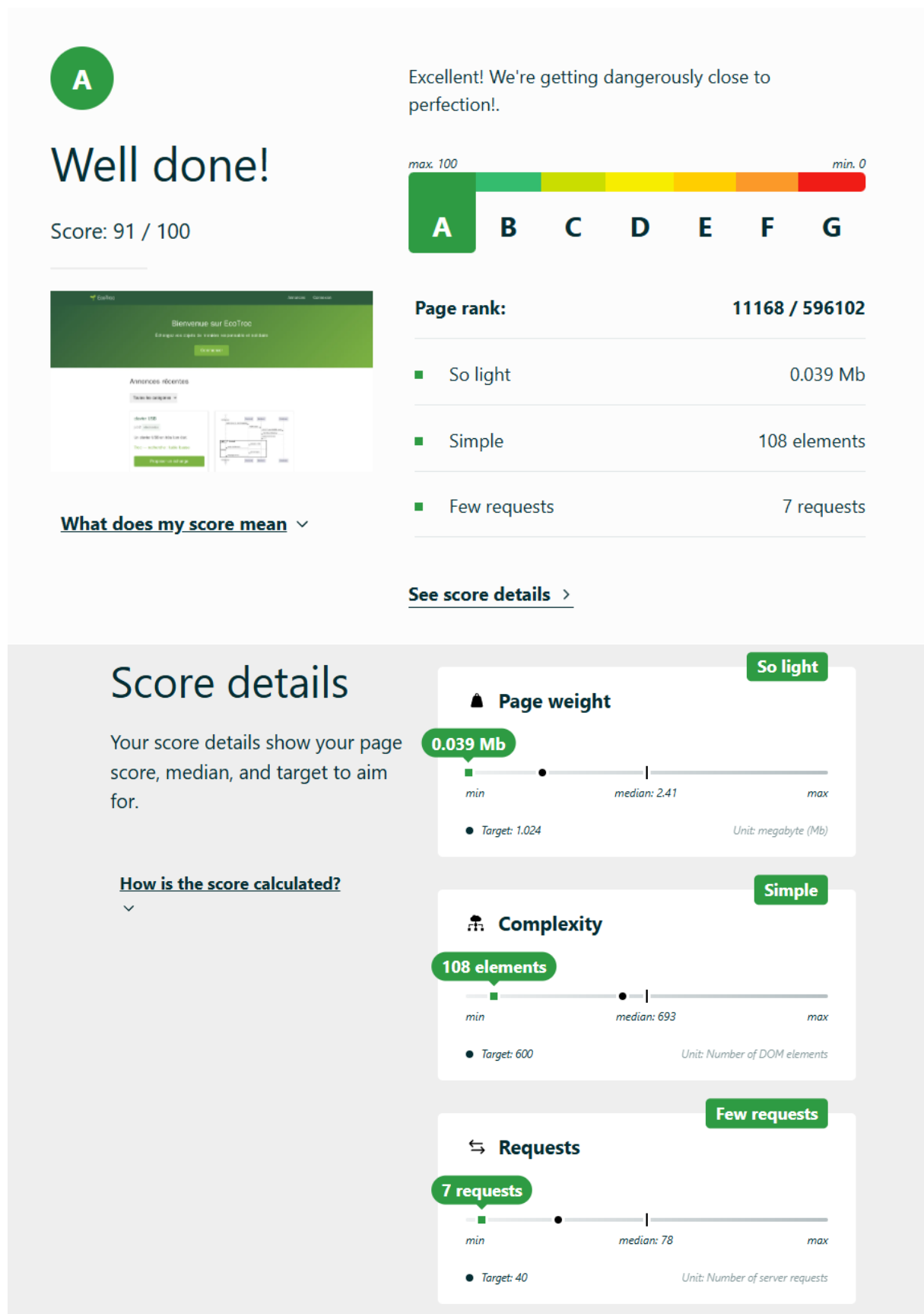
- Application légère (9 Ko, 5 requêtes)
- Architecture simple sans dépendance frontend
- Score Lighthouse 100
- Système d'échange complet avec chat et double confirmation

Améliorations possibles :

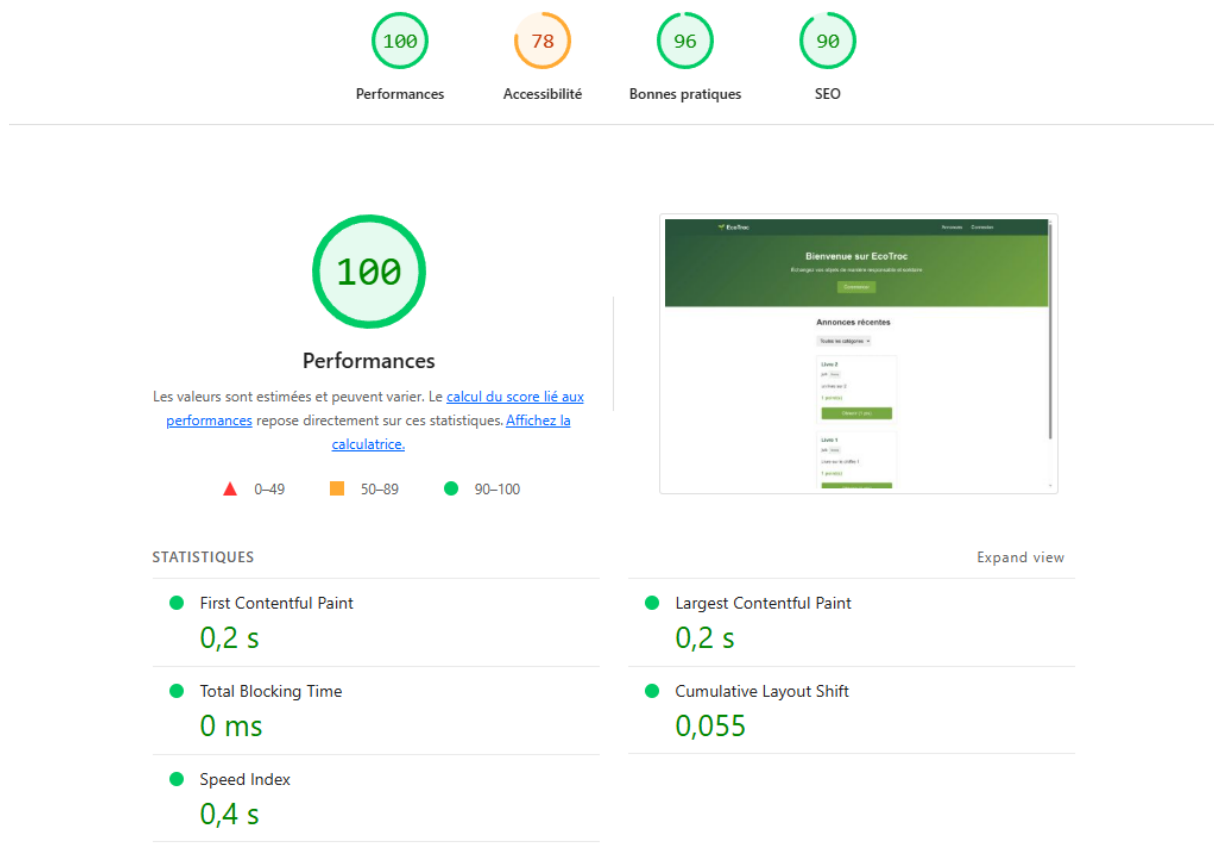
- Notifications en temps réel (WebSocket plutôt que polling)
- Système de notation des utilisateurs
- Application mobile

9 Outils de test

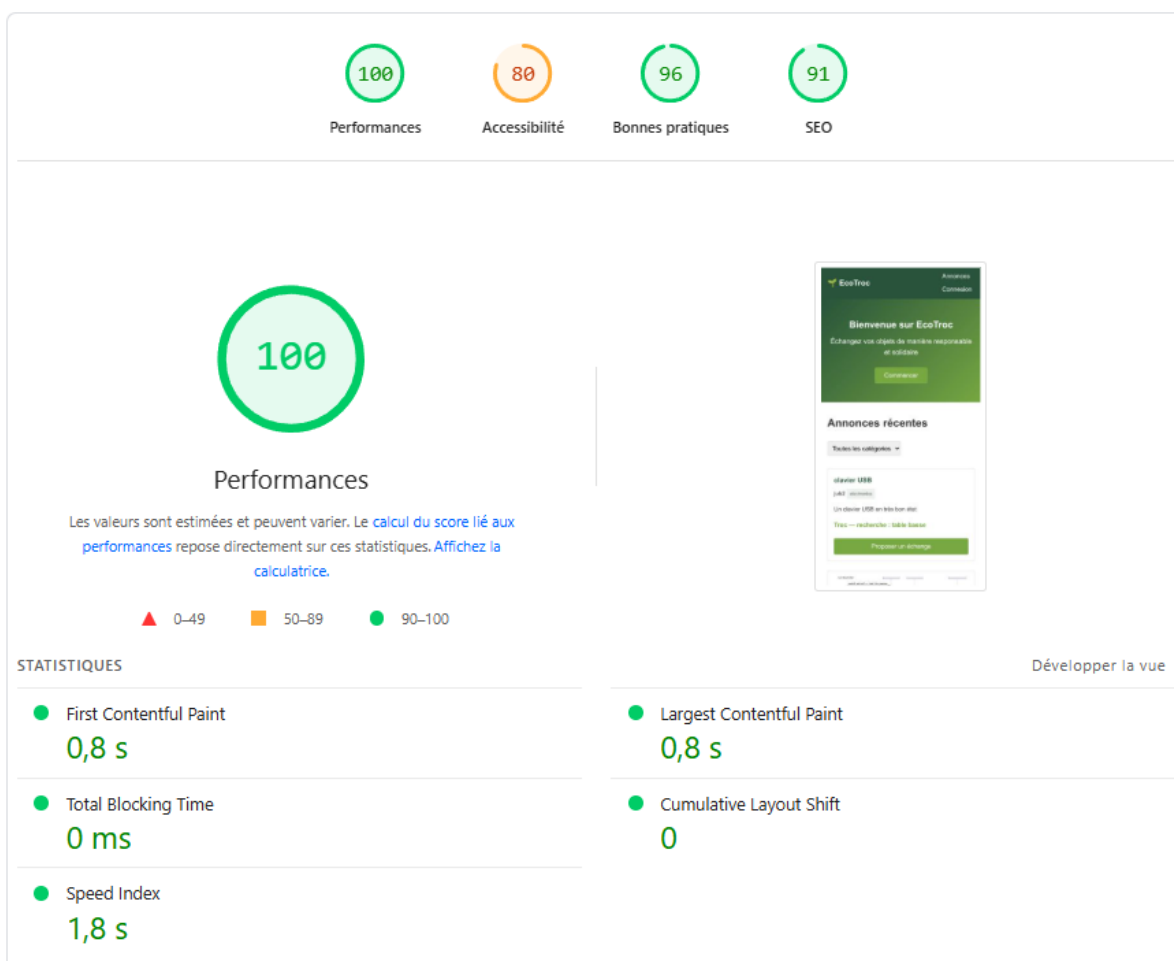
9.1 EcoIndex



9.2 Lighthouse



9.3 Page Speed Insights



9.4 CatchPoint

